

Simulation for PseudoBulk Benchmarking

Léopold Guyot

2025-05-22

Idea

- Subtype effect determined at the patient level?
- Currently a mix of the two, not ideal
- Remove subtype effect ## Packages

```
library(scpdata)
library(scp)
library(SingleCellExperiment)
library(msqrob2)
library(iCOBRA)
library(UpSetR)
library(tidyverse)
```

Simulation strategy

Experimental Data

To generate our data, we will use the dataset obtained from Brunner et al. 2022. (Label-Free). We will isolate only one condition (Untreated). For this condition we compute for each protein; his mean log-intensity and his standard deviation

Intensities modelling

Let X_{ij} denote the log-intensity of protein i in cell j .

We model the log-intensity as:

$$X_{ij} = \mu_i + \epsilon_{ij} + A_i^{\text{patient}} \cdot \delta_{i,p_j}^{\text{patient}} + A_i^{\text{subtype}} \cdot \delta_{i,s_j}^{\text{subtype}} + A_i^{\text{treatment}} \cdot \delta_i^{\text{treatment}} \cdot \mathbb{I}_{t_j=1}$$

Where:

- μ_i : baseline log-intensity of protein i obtained from experimental data
- $\epsilon_{ij} \sim \mathcal{N}(0, \sigma_i^2)$: residual noise obtained from experimental data
- A_i^{patient} : indicator if protein i is affected by patient-specific shifts
- A_i^{subtype} : indicator if protein i is affected by subtype-specific shifts
- $A_i^{\text{treatment}}$: indicator if protein i is affected by treatment
- $p_j \in \{1, \dots, P\}$: patient identity of cell j

- $s_j \in \{1, \dots, S\}$: subtype identity of cell j
- $t_j \in \{0, 1\}$: treatment status of cell j (1 = treated)
- $\delta_{i,p}^{\text{patient}} \sim \mathcal{N}(0, \sigma_{\text{patient}}^2)$: patient-specific shift for protein i and patient p
- $\delta_{i,s}^{\text{subtype}} \sim \mathcal{N}(0, \sigma_{\text{subtype}}^2)$: subtype-specific shift for protein i and subtype s
- $\delta_i^{\text{treatment}} \sim \mathcal{N}(0, \sigma_{\text{treatment}}^2)$: treatment effect for protein i
- $\mathbb{I}_{t_j=1}$: indicator function equals 1 if cell j is treated, 0 otherwise

Missing Data Modelling

Each X_{ij} is observed with probability $1 - \pi_i$, where:

$$M_{ij} \sim \text{Bernoulli}(\pi_i)$$

If $M_{ij} = 1$, then $X_{ij} = \text{NA}$.

Where π_i is the experimentally observed missing rate of protein i .

Simulation parameters

```
# number of patients
nPatient = 16

# Number of cell population
nCellSubtype <- 5

# Cells by patient/subtype
nCellPatientSub = 50

subtypeNames <- paste0("Type", seq_len(nCellSubtype))

nCellTot = nPatient * nCellSubtype * nCellPatientSub

# Inter-patient variability
patientSD <- 0.5

# Prop of proteins shifted by patient effect
patientEffect <- 0.7

# Inter-population variability
subtypeSD <- 1

# Prop of proteins shifted by subtype effect
subtypeEffect <- 0.4

# proportion of proteins affected by treatment
treatmentEffect <- 0.3
```

```
# treatment shift strength
treatmentStrength <- 1
```

Experimental Data

Prepare base data obtained from Brunner et al. 2022: Single cell proteomics data acquired by the Mann Lab using a newly designed timsTOF instrument, referred to as timsTOF-SCP. The dataset contains quantitative information from single-cells blocked at 4 cell cycle stages: G1, G1-S, G2, G2-M. The data was acquired using a label-free sample preparation protocols combined to a data independent (DIA) acquisition mode.

We subset the PSM data for the Untreated cell group. Then we apply a log2 transformation to have the intensities distributed normally.

```
base <- scpdata::brunner2022()
```

```
## see ?scpdata and browseVignettes('scpdata') for documentation
```

```
## loading from cache
```

```
sce <- getWithColData(base, "proteins")
sce <- sce[, colData(sce)$CellCycleStage == "UB"]
sce <- logTransform(sce) # log transform to obtain normal intensities distribution
sce <- sce[rowSums(!is.na(assay(sce))) > 1, ]
sce
```

```
## class: SingleCellExperiment
```

```
## dim: 2373 203
```

```
## metadata(0):
```

```
## assays(1): ''
```

```
## rownames(2373): UBA6_HUMAN SHOT1_HUMAN ... NS1BP_HUMAN S23IP_HUMAN
```

```
## rowData names(5): Protein.Group Protein.Ids Protein.Names Genes
```

```
## First.Protein.Description
```

```
## colnames(203):
```

```
## D:\Andreas_Brunner\Projects\SingleCellProteomics\202109_WorkHere\SingleCells\SingleShots_RawData\2
```

```
## D:\Andreas_Brunner\Projects\SingleCellProteomics\202109_WorkHere\SingleCells\SingleShots_RawData\2
```

. . .

```
## D:\Andreas_Brunner\Projects\SingleCellProteomics\202109_WorkHere\SingleCells\SingleShots_RawData\2
```

```
## D:\Andreas_Brunner\Projects\SingleCellProteomics\202109_WorkHere\SingleCells\SingleShots_RawData\2
```

```
## colData names(14): File.Name Run ... RunID Label
```

```
## reducedDimNames(0):
```

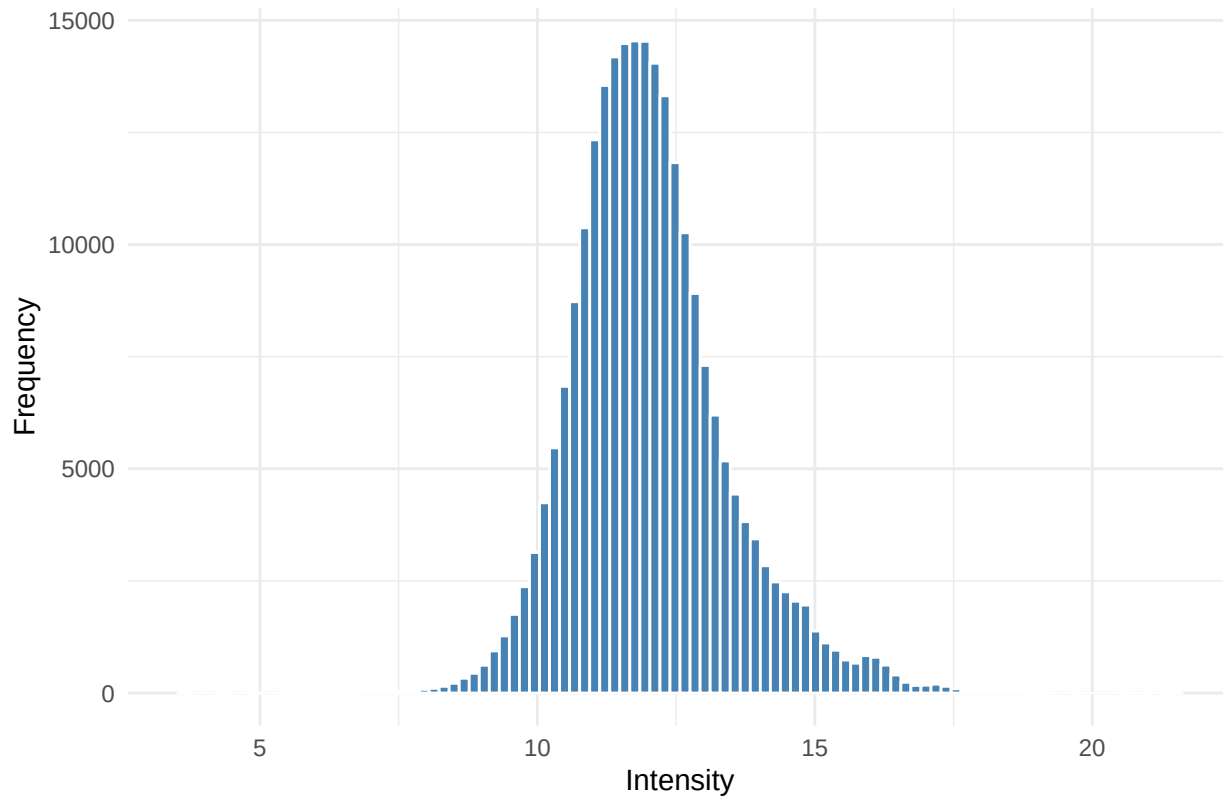
```
## mainExpName: NULL
```

```
## altExpNames(0):
```

Intensity distribution

```
int_mat <- assay(sce)
int_vec <- as.vector(int_mat)
int_vec <- int_vec[!is.na(int_vec)]
ggplot(data.frame(intensity = int_vec), aes(x = intensity)) +
  geom_histogram(bins = 100, fill = "steelblue", color = "white") +
  theme_minimal() +
  labs(title = "Distribution of protein log-intensities across all cells",
       x = "Intensity", y = "Frequency")
```

Distribution of protein log-intensities across all cells



Get log-intensities metrics

```
protAssay <- as.data.frame(assay(sce))
```

Compute metric

```
meanIntensity <- rowMeans(protAssay, na.rm = TRUE)
```

```
sdIntensity <- apply(protAssay, 1, sd, na.rm = TRUE)
```

```
pctMissing <- rowMeans(is.na(protAssay)) * 100
```

```
protMetrics <- data.frame(
  protNames = rownames(protAssay),
  meanIntensity = meanIntensity,
  sdIntensity = sdIntensity,
  pctMissing = pctMissing,
  stringsAsFactors = FALSE
)
```

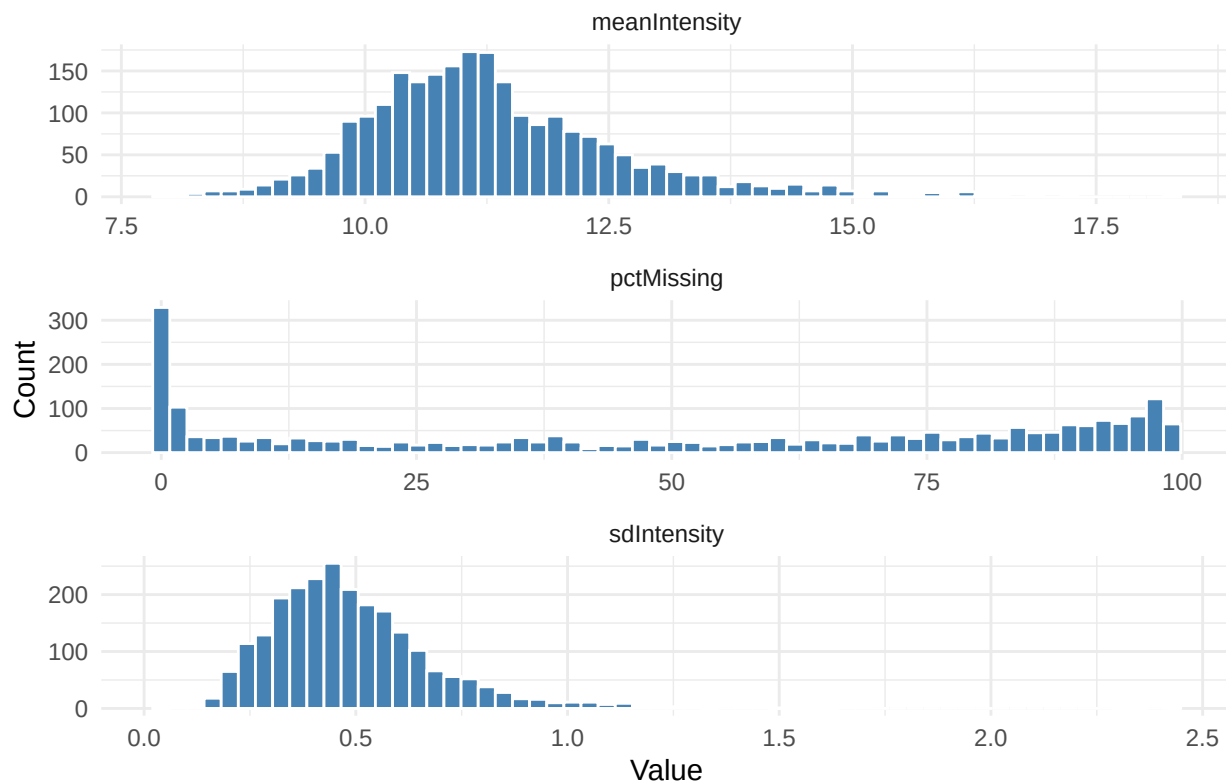
```
head(protMetrics)
```

##	protNames	meanIntensity	sdIntensity	pctMissing
##	UBA6_HUMAN UBA6_HUMAN	10.80805	0.3678057	95.56650
##	SHOT1_HUMAN SHOT1_HUMAN	10.53306	0.5743521	76.35468
##	ILVBL_HUMAN ILVBL_HUMAN	10.10614	0.8832194	97.04433
##	VWA8_HUMAN VWA8_HUMAN	10.73573	0.4490042	92.61084
##	GTPBA_HUMAN GTPBA_HUMAN	10.13827	0.7263385	85.71429
##	CNOT1_HUMAN CNOT1_HUMAN	11.17584	0.5817081	50.73892

Intensities metrics distribution

```
protMetrics_long <- protMetrics %>%  
  pivot_longer(cols = c(meanIntensity, sdIntensity, pctMissing),  
    names_to = "metric",  
    values_to = "value")  
  
ggplot(protMetrics_long, aes(x = value)) +  
  geom_histogram(bins = 60, fill = "steelblue", color = "white") +  
  facet_wrap(~ metric, scales = "free", ncol = 1) +  
  theme_minimal() +  
  labs(title = "Distribution of per-protein metrics",  
    x = "Value", y = "Count")
```

Distribution of per-protein metrics



Simulation (patient and subtype pattern)

```
mu <- protMetrics$meanIntensity  
proteinNames <- protMetrics$protNames  
residualSD <- protMetrics$sdIntensity  
missingProb <- protMetrics$pctMissing/100  
nProt <- length(mu)  
  
# Initialize matrix  
simMatrix <- matrix(NA, nrow = nProt, ncol = nCellTot)  
rownames(simMatrix) <- proteinNames  
cellIds <- paste0("P", rep(1:nPatient, each = nCellPatientSub), "_C", 1:nCellTot)  
colnames(simMatrix) <- cellIds
```

```

subtypeEffects <- matrix(
  rnorm(nProt * nCellSubtype, mean = 0, sd = subtypeSD),
  nrow = nProt,
  ncol = nCellSubtype,
  dimnames = list(proteinNames, subtypeNames)
)

nPatientShiftProt <- floor(patientEffect * nProt)
nSubtypeShiftProt <- floor(subtypeEffect * nProt)

# Prepare colData dataframe
subtypesPerPatient <- rep(subtypeNames, each = nCellPatientSub)
allSubtypes <- rep(subtypesPerPatient, times = nPatient)

patients <- sub("_.*", "", colnames(simMatrix))

colData <- data.frame(
  cellId = colnames(simMatrix),
  patient = patients,
  subtype = allSubtypes,
  patientShift = numeric(nCellTot),
  subtypeShift = numeric(nCellTot),
  stringsAsFactors = FALSE
)

cellCounter <- 1

# Simulate
for (i in 1:nPatient) {
  patientShift <- numeric(nProt)
  names(patientShift) <- proteinNames
  # Choose which proteins will receive the shift
  patientShiftProteins <- sample(proteinNames, nPatientShiftProt)
  patientShift[patientShiftProteins] <- rnorm(nPatientShiftProt, mean = 0, sd = patientSD)

  for (s in 1:nCellSubtype) {
    subtypeShiftProteins <- sample(proteinNames, nSubtypeShiftProt)
    subtypeName <- subtypeNames[s]
    # Extract subtype effect for all proteins
    subtypeShiftFull <- subtypeEffects[, subtypeName]

    # Zero out subtype shifts for proteins NOT in the sampled subset
    subtypeShift <- numeric(nProt)
    names(subtypeShift) <- proteinNames
    subtypeShift[subtypeShiftProteins] <- subtypeShiftFull[subtypeShiftProteins]
    for (k in 1:nCellPatientSub) {
      cellIndex <- cellCounter
      colData$patientShift[cellIndex] <- list(patientShift)
      colData$subtypeShift[cellIndex] <- list(subtypeShift)
      simMatrix[, cellIndex] <- rnorm(nProt, mean = mu, sd = residualSD) + patientShift + subtypeShift

      colData$subtype[cellIndex] <- subtypeName

      cellCounter <- cellCounter + 1
    }
  }
}

```

```

    }
  }
}

```

Simulate data missingness

```

# Add missing values

for (p in seq_len(nProt)) {
  missing <- runif(nCellTot) < missingProb[p]
  simMatrix[p, missing] <- NA
}

```

Should the sampling of shifted proteins be the same for all patients/celltypes?

Create the SingleCellExperiment object

```

simSCE <- SingleCellExperiment(assays = SimpleList(simMatrix),
                              rowData = rowData(sce),
                              colData = colData)

simSCE

## class: SingleCellExperiment
## dim: 2373 4000
## metadata(0):
## assays(1): ''
## rownames(2373): UBA6_HUMAN SHOT1_HUMAN ... NS1BP_HUMAN S23IP_HUMAN
## rowData names(5): Protein.Group Protein.Ids Protein.Names Genes
##   First.Protein.Description
## colnames(4000): P1_C1 P1_C2 ... P16_C3999 P16_C4000
## colData names(5): cellId patient subtype patientShift subtypeShift
## reducedDimNames(0):
## mainExpName: NULL
## altExpNames(0):

```

Add treatment effect

```

# Initialize treatment vector
treatment_vector <- character(ncol(simSCE))
names(treatment_vector) <- colnames(simSCE)

# Get colData
cd <- colData(simSCE)

# Loop over each patient-subtype group
for (p in unique(cd$patient)) {
  for (s in unique(cd$subtype)) {
    # Find cell indices in this group
    group_cells <- which(cd$patient == p & cd$subtype == s)

    # Shuffle and assign 50% treated, 50% control
    n_group <- length(group_cells)
    n_treat <- floor(n_group / 2)
  }
}

```

```

    treated_cells <- sample(group_cells, size = n_treat)
    control_cells <- setdiff(group_cells, treated_cells)

    treatment_vector[treated_cells] <- "treated"
    treatment_vector[control_cells] <- "control"
  }
}

# Assign to colData
colData(simSCE)$treatment <- treatment_vector

# Select affected proteins
nProt <- nrow(simSCE)
proteinNames <- rownames(simSCE)
nAffected <- floor(treatmentEffect * nProt)
affected_proteins <- sample(proteinNames, size = nAffected, replace = FALSE)

# Simulate shift per affected protein
treatment_shift_vec <- numeric(nProt)
names(treatment_shift_vec) <- proteinNames
treatment_shift_vec[affected_proteins] <- rnorm(nAffected, mean = 0, sd = treatmentStrength)

# Modify assay
exprMat <- assay(simSCE)
treated_cells <- colData(simSCE)$treatment == "treated"

# Apply shift: add the same shift for each affected protein across treated cells
exprMat[affected_proteins, treated_cells] <- exprMat[affected_proteins, treated_cells] +
  treatment_shift_vec[affected_proteins]

# Save modified matrix back
assay(simSCE) <- exprMat

rowData(simSCE)$treatmentAffected <- rownames(simSCE) %in% affected_proteins
rowData(simSCE)$treatmentShift <- treatment_shift_vec

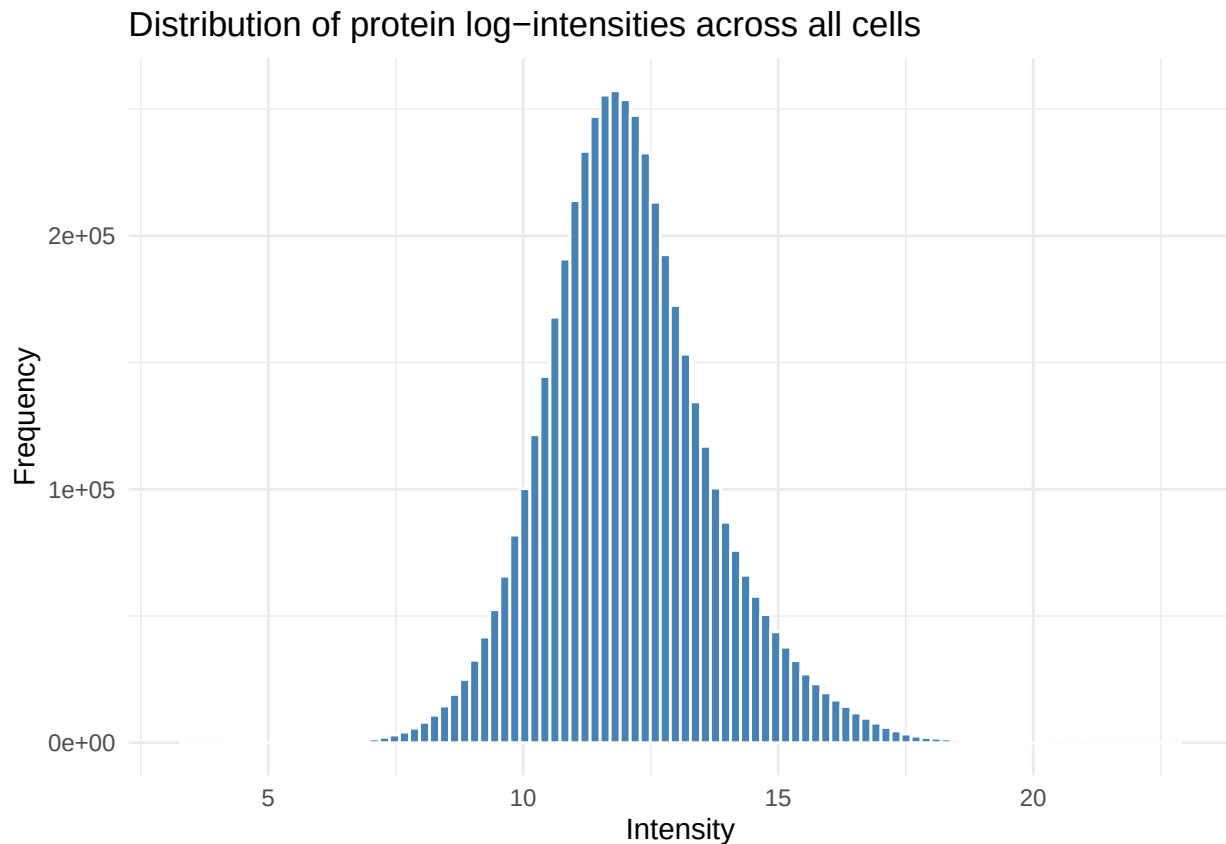
simSCE

## class: SingleCellExperiment
## dim: 2373 4000
## metadata(0):
## assays(1): '
## rownames(2373): UBA6_HUMAN SHOT1_HUMAN ... NS1BP_HUMAN S23IP_HUMAN
## rowData names(7): Protein.Group Protein.Ids ... treatmentAffected
##   treatmentShift
## colnames(4000): P1_C1 P1_C2 ... P16_C3999 P16_C4000
## colData names(6): cellId patient ... subtypeShift treatment
## reducedDimNames(0):
## mainExpName: NULL
## altExpNames(0):

```


Plot new distribution of log-intensities

```
int_mat <- assay(simSCE)
int_vec <- as.vector(int_mat)
int_vec <- int_vec[!is.na(int_vec)]
ggplot(data.frame(intensity = int_vec), aes(x = intensity)) +
  geom_histogram(bins = 100, fill = "steelblue", color = "white") +
  theme_minimal() +
  labs(title = "Distribution of protein log-intensities across all cells",
       x = "Intensity", y = "Frequency")
```



Apply scplainer

```
scpSIM <- scpModelWorkflow(simSCE, formula = ~ 1 + patient + subtype + treatment, verbose = FALSE)
scpRes <- scpDifferentialAnalysis(
  scpSIM,
  contrasts = list(c("treatment", "control", "treated"))
)[[1]]
```

Apply msqrob2

```
L <- makeContrast("treatmenttreated=0", parameterNames = c("treatmenttreated"))

msqSIM <- suppressWarnings(msqrob(simSCE, formula = ~ 1 + patient + subtype + treatment))
msqRes <- rowData(hypothesisTest(object = msqSIM, contrast = L))$treatmenttreated
```

Apply PseudoBulk + msqrob2

```
aggregate_se <- function(se, group_by_cols, fun = mean) {
  if (!all(group_by_cols %in% colnames(colData(se)))) {
    stop("Some specified group_by_cols do not exist in colData.")
  }
  colData(se)$Group <- apply(colData(se)[, group_by_cols, drop = FALSE], 1, paste, collapse = "_")
  group_factor <- factor(colData(se)$Group)
  assay_matrix <- assay(se)

  # Aggregate assay matrix by group_factor levels order
  grouped_indices <- split(seq_along(group_factor), group_factor)
  levels_group <- levels(group_factor) # factor levels in order
  aggregated_assay <- do.call(cbind, lapply(levels_group,
                                           function(g) {
                                             idx <- grouped_indices[[g]]
                                             apply(assay_matrix[, idx, drop = FALSE], 1, fun, na.rm = TRUE)
                                           }))

  new_colData <- colData(se) %>%
    as.data.frame() %>%
    group_by(Group) %>%
    summarise(across(all_of(group_by_cols), dplyr::first), .groups = "drop") %>%
    as.data.frame()

  # Reorder new_colData rows to match factor levels
  new_colData <- new_colData[match(levels_group, new_colData$Group), ]
  rownames(new_colData) <- new_colData$Group
  new_se <- SummarizedExperiment(assays = list(aggregated_assay),
                                colData = new_colData,
                                rowData = rowData(se))

  return(new_se)
}

pbSIM <- aggregate_se(simSCE, c("patient", "subtype", "treatment"), fun = mean)
pbSIM <- suppressWarnings(msqrob(pbSIM, ~ 1 + patient + subtype + treatment))
pbRes <- rowData(hypothesisTest(object = pbSIM, contrast = L))$treatmenttreated
```

Get FDR curves

```
# Extract p-values from each result
pval_pb <- pbRes[, "pval", drop = FALSE]
colnames(pval_pb) <- "PBmsqrob2"

pval_msq <- msqRes[, "pval", drop = FALSE]
colnames(pval_msq) <- "msqrob2"

pval_scp <- as.data.frame(scpRes[, c("feature", "pvalue")])
colnames(pval_scp) <- c("feature", "scplainer")
rownames(pval_scp) <- pval_scp$feature
pval_scp$feature <- NULL

# Merge all three by rownames (feature names)
tmp1 <- merge(pval_pb, pval_msq, by = "row.names", all = TRUE)
```

```

rownames(tmp1) <- tmp1$Row.names
tmp1$Row.names <- NULL

combined_pvals <- merge(tmp1, pval_scp, by = "row.names", all = TRUE)
rownames(combined_pvals) <- combined_pvals$Row.names
combined_pvals$Row.names <- NULL

# Extract adjusted p-values from each result
padj_pb <- pbRes[, "adjPval", drop = FALSE]
colnames(padj_pb) <- "PBmsqrob2"

padj_msq <- msqRes[, "adjPval", drop = FALSE]
colnames(padj_msq) <- "msqrob2"

padj_scp <- as.data.frame(scpRes[, c("feature", "padj")])
colnames(padj_scp) <- c("feature", "scplainer")
rownames(padj_scp) <- padj_scp$feature
padj_scp$feature <- NULL

# Merge all three by rownames (feature names)
tmp_padj <- merge(padj_pb, padj_msq, by = "row.names", all = TRUE)
rownames(tmp_padj) <- tmp_padj$Row.names
tmp_padj$Row.names <- NULL

combined_padj <- merge(tmp_padj, padj_scp, by = "row.names", all = TRUE)
rownames(combined_padj) <- combined_padj$Row.names
combined_padj$Row.names <- NULL

truth <- data.frame(isTreated = rowData(simSCE)$treatmentAffected,
                    row.names = rownames(rowData(simSCE))
                    )

cobData <- COBRADData(
  pval = combined_pvals,
  padj = combined_padj,
  truth = truth
)

perfRes <- calculate_performance(cobradata = cobData,
                               binary_truth = "isTreated") %>%
  fdrtpr() %>%
  mutate(thr = as.numeric(sub("thr", "", thr)))

## binary_truth column is logical, converting to numeric (0/1)
## Object up to date

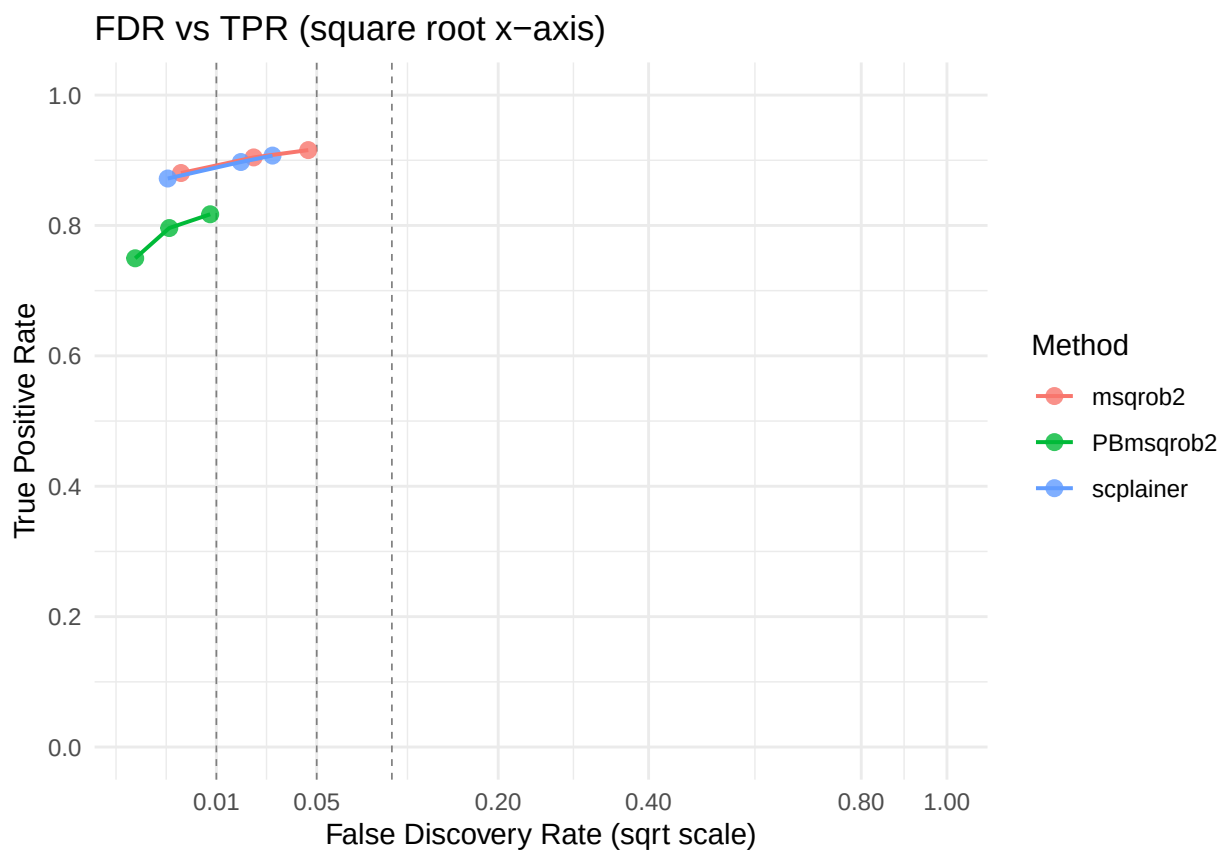
plot <- ggplot(perfRes, aes(x = FDR, y = TPR, color = method)) +
  geom_vline(
    xintercept = c(0.01, 0.05, 0.1),
    linetype = "dashed", color = "grey50", size = 0.3
  ) +
  geom_point(size = 2.5, alpha = 0.8) +
  geom_line(size = 0.7) +
  scale_x_continuous(

```

```

    trans = "sqrt",
    limits = c(0, 1),
    breaks = c(0.01, 0.05, 0.2, 0.4, 0.8, 1),
    labels = scales::label_number()
  ) +
  scale_y_continuous(
    limits = c(0, 1),
    breaks = seq(0, 1, 0.2)
  ) +
  theme_minimal() +
  labs(
    title = "FDR vs TPR (square root x-axis)",
    x = "False Discovery Rate (sqrt scale)",
    y = "True Positive Rate",
    color = "Method"
  )
)
plot

```



Upset plot

```

scpFeatures <- scpRes %>%
  as.data.frame() %>%
  filter(padj <= 0.05) %>%
  pull(feature)

msqFeatures <- msqRes %>%

```

```

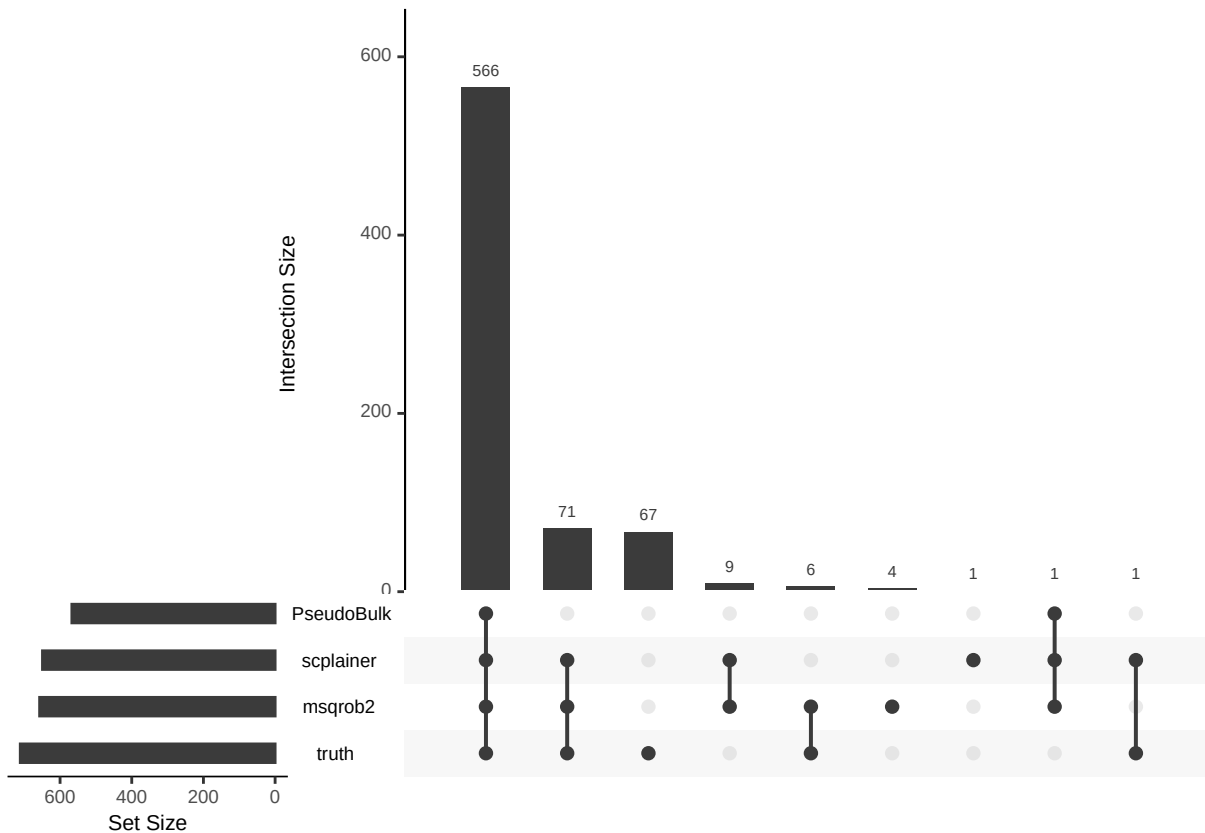
as.data.frame() %>%
filter(adjPval <= 0.05) %>%
rownames()

pbFeatures <- pbRes %>%
as.data.frame() %>%
filter(adjPval <= 0.05) %>%
rownames()

truthFeatures <- rowData(simSCE) %>%
as.data.frame() %>%
filter(treatmentAffected) %>%
rownames()

listUpSet <- list("scplainer" = scpFeatures,
                 "msqrob2" = msqFeatures,
                 "PseudoBulk" = pbFeatures,
                 "truth" = truthFeatures)
upset(fromList(listUpSet), order.by = "freq")

```



```

sessionInfo()

## R version 4.5.0 (2025-04-11)
## Platform: x86_64-pc-linux-gnu
## Running under: Pop!_OS 24.04 LTS
##
## Matrix products: default
## BLAS: /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0

```

```

## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0  LAPACK version 3.12.0
##
## locale:
## [1] LC_CTYPE=en_GB.UTF-8      LC_NUMERIC=C
## [3] LC_TIME=en_GB.UTF-8      LC_COLLATE=en_GB.UTF-8
## [5] LC_MONETARY=en_GB.UTF-8  LC_MESSAGES=en_GB.UTF-8
## [7] LC_PAPER=en_GB.UTF-8     LC_NAME=C
## [9] LC_ADDRESS=C             LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_GB.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Europe/Brussels
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] lubridate_1.9.4          forcats_1.0.0
## [3] stringr_1.5.1           dplyr_1.1.4
## [5] purrr_1.0.4             readr_2.1.5
## [7] tidyr_1.3.1             tibble_3.2.1
## [9] ggplot2_3.5.2           tidyverse_2.0.0
## [11] UpSetR_1.4.0            iCOBRA_1.36.0
## [13] msqrob2_1.16.0          SingleCellExperiment_1.30.1
## [15] scp_1.18.0              scpdata_1.16.1
## [17] ExperimentHub_2.16.0    AnnotationHub_3.16.0
## [19] BiocFileCache_2.16.0    dbplyr_2.5.0
## [21] QFeatures_1.19.1       MultiAssayExperiment_1.34.0
## [23] SummarizedExperiment_1.38.1 Biobase_2.68.0
## [25] GenomicRanges_1.60.0    GenomeInfoDb_1.44.0
## [27] IRanges_2.42.0          S4Vectors_0.46.0
## [29] BiocGenerics_0.54.0     generics_0.1.4
## [31] MatrixGenerics_1.20.0   matrixStats_1.5.0
## [33] rmarkdown_2.29
##
## loaded via a namespace (and not attached):
## [1] RColorBrewer_1.1-3      rstudioapi_0.17.1      jsonlite_2.0.0
## [4] magrittr_2.0.3          farver_2.1.2           nloptr_2.2.1
## [7] vctrs_0.6.5            ROCR_1.0-11            memoise_2.0.1
## [10] minqa_1.2.8            tinytex_0.57           htmltools_0.5.8.1
## [13] S4Arrays_1.8.0         BiocBaseUtils_1.10.0   curl_6.2.2
## [16] SparseArray_1.8.0      praca_2.4.4            htmlwidgets_1.6.4
## [19] plyr_1.8.9             cachem_1.1.0           igraph_2.1.4
## [22] mime_0.13              lifecycle_1.0.4        pkgconfig_2.0.3
## [25] optimx_2025-4.9        Matrix_1.7-3           R6_2.6.1
## [28] fastmap_1.2.0          GenomeInfoDbData_1.2.14 rbibutils_2.3
## [31] shiny_1.10.0           clue_0.3-66            numDeriv_2016.8-1.1
## [34] digest_0.6.37          fdrtool_1.2.18         AnnotationDbi_1.70.0
## [37] lpsymphony_1.36.0      RSQLite_2.3.11         labeling_0.4.3
## [40] filelock_1.0.3         timechange_0.3.0       httr_1.4.7
## [43] abind_1.4-8            compiler_4.5.0         withr_3.0.2
## [46] bit64_4.6.0-1          BiocParallel_1.42.0    DBI_1.2.3
## [49] MASS_7.3-65            rappdirs_0.3.3         DelayedArray_0.34.1

```

## [52] tools_4.5.0	httpuv_1.6.16	glue_1.8.0
## [55] nlme_3.1-168	promises_1.3.2	grid_4.5.0
## [58] cluster_2.1.8.1	reshape2_1.4.4	gtable_0.3.6
## [61] tzdb_0.5.0	shinyBS_0.61.1	hms_1.1.3
## [64] metapod_1.16.0	XVector_0.48.0	ggrepel_0.9.6
## [67] BiocVersion_3.21.1	pillar_1.10.2	limma_3.64.0
## [70] later_1.4.2	splines_4.5.0	lattice_0.22-5
## [73] bit_4.6.0	tidyselect_1.2.1	Biostrings_2.76.0
## [76] knitr_1.50	reformulas_0.4.1	gridExtra_2.3
## [79] ProtGenerics_1.40.0	IHW_1.36.0	xfun_0.52
## [82] shinydashboard_0.7.3	statmod_1.5.0	DT_0.33
## [85] stringi_1.8.7	UCSC.utils_1.4.0	lazyeval_0.2.2
## [88] yaml_2.3.10	boot_1.3-31	evaluate_1.0.3
## [91] codetools_0.2-20	nipals_1.0	MsCoreUtils_1.20.0
## [94] BiocManager_1.30.25	cli_3.6.5	xtable_1.8-4
## [97] Rdpack_2.6.4	Rcpp_1.0.14	png_0.1-8
## [100] parallel_4.5.0	blob_1.2.4	AnnotationFilter_1.32.0
## [103] lme4_1.1-37	slam_0.1-55	scales_1.4.0
## [106] crayon_1.5.3	rlang_1.1.6	KEGGREST_1.48.0